DAY 16 | POST-READ

Spark, Pipelines and SQL: Recap and Beyond

What we covered today, plus ideas to stretch your understanding.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Part A: Summary of today

Topic	What to remember
Medallion layers	Bronze raw, silver clean, gold aggregated. Each rebuildable from the previous.
Production batch jobs	Scheduled, idempotent, partitioned, observable.
Spark architecture	Driver plans; executors run tasks on partitions in parallel.
Lazy evaluation	Transformations build a plan; actions (show, count, write) run it.
Transformations	select, filter, withColumn, when/otherwise, dropDuplicates, groupBy().agg().
Joins	inner, left, left_anti, left_semi. Check row counts after joining.
SQL and Spark SQL	WHERE before groups, HAVING after; CTEs, windows; temp views + spark.sql.
Production pipeline	Run date parameter, quality checks, overwrite per partition, run log.
AI-assisted DE (NEW v4)	Specify rules, let Codex draft, verify with golden examples, counts and totals.

Part B: Going beyond today

1. Explicit schemas

inferSchema reads the data twice and can guess wrong. Production jobs declare the schema.

```
schema = "order_id INT, customer_id INT, amount DOUBLE, order_date DATE, city STRING"
df = spark.read.csv("orders.csv", header=True, schema=schema)
```

2. Shuffles and the plan

groupBy and joins move data between executors so matching keys meet. This shuffle is the most expensive part of most jobs. Filter early and select only needed columns. Use df.explain() to see the plan.

3. Dynamic partition overwrite

Only the partitions present in the new data are replaced, so re-running one day does not delete the others.

```
spark.conf.set("spark.sql.sources.partitionOverwriteMode", "dynamic")
gold.write.mode("overwrite").partitionBy("order_date").parquet(path)
```

4. Scheduling and Delta Lake

Real pipelines run from a scheduler (cron, Apache Airflow, Azure Data Factory or Databricks Jobs) that passes the run date, retries and alerts. Delta Lake adds transactions, updates and time travel on top of Parquet.

5. Reviewing AI-written SQL

Ask Codex to add the business question as a comment above each query, then check the query answers that question. Common AI mistakes: an inner join that silently drops unknown customers, a filter in WHERE that belonged in HAVING, and averages of averages.

Part C: Check yourself

- What do the driver and executors each do?
- Why does df.filter(...) return instantly even on a huge file?
- When would you use a left_anti join?
- Why should a pipeline take the run date as a parameter?
- Name three ways to verify a transformation that Codex wrote.

Part D: Practice ideas for tonight

- Add a LAG-based day-over-day revenue change to the gold layer.
- Ask Codex for a new gold aggregation, then write a golden-example test for it.
- Use a window function to find the slowest-resolved ticket per priority.

Coming up on Day 17: DevOps and cloud fundamentals, DevSecOps, Azure DevOps, and Basics assessment 3.

Otsukaresama deshita (お疲れ様でした): thank you for your hard work today